messages to that actor or another actor. You need to verify that this stream of messages is as expected.

So far, you've only used a probe to check whether a message was forwarded to that probe by a real actor. For example, when you forwarded in `SimplifiedManager`,

```
          case Forward(text, sendTo) =>
            sendTo ! text
```

`Forward` was the protocol of the real actor, and `sentTo` was the probe. You sent a message to the real actor and checked with the probe to see if the message was finally forwarded.

The idea now is to place a probe in front of the actor under test so that the probe receives the message and then passes it to the actor behind it. Although the protocol is from the behavior under test, you interact with this monitor as if it were the actor under test. This is done with the following `monitor` method from the `Behaviors` factory:

```
def monitor(monitor: ActorRef[T], behavior: Behavior[T]): Behavior[T]
```

Here, `T` is the protocol of the actor under test, and `monitor` is the probe—`probe.ref`, of type `ActorRef`. You can see this in practice in the following simple test:

```
class AsyncForwardSpec ...
  ...

  "a monitor" must {

    "intercept the messages" in {

      val probe = createTestProbe[String]
      val behaviorUnderTest = Behaviors.receiveMessage[String] { _ =>
        Behaviors.ignore
      }
      val behaviorMonitored = Behaviors.monitor(probe.ref, behaviorUnderTest)
❶

      val actor = testkit.spawn(behaviorMonitored)

      actor ! "checking"
      probe.expectMessage("checking")

    }
  }
```